

Q1. Compare object-oriented and functional paradigms using Java.
Consider computing the squares of a list of numbers, shown in both styles.

Object-oriented style:

```
class SquareCalculator {
    int square(int x) {
        return x * x;
    }
}

public class OopStyle {
    public static void main(String[] args) {
        SquareCalculator calc = new SquareCalculator();
        int[] nums = {1, 2, 3, 4};
        for (int n : nums) {
            System.out.println(calc.square(n));
        }
    }
}
```

Functional style:

```
import java.util.List;
import java.util.stream.Collectors;

public class FunctionalStyle {
    public static void main(String[] args) {
        List<Integer> nums = List.of(1, 2, 3, 4);
        List<Integer> squares = nums.stream()
            .map(x -> x * x)
            .collect(Collectors.toList());
        System.out.println(squares);
    }
}
```

The object-oriented version encapsulates behavior inside an object (calc) and iterates explicitly, relying on state and control flow. The functional version expresses the transformation directly using a map operation with a lambda, without creating any object to hold the behavior, and without an explicit loop or mutable variable.

Q2. Demonstrate how a problem is solved differently in procedural and object-oriented Java code.

Consider the problem of managing a simple list of book titles and printing them.

Procedural solution:

```
public class BooksProcedural {
    static String[] titles = {"Java Basics", "Data Structures"};

    static void printTitles() {
        for (String t : titles) {
            System.out.println(t);
        }
    }

    public static void main(String[] args) {
        printTitles();
    }
}
```

Object-oriented solution:

```
import java.util.ArrayList;
import java.util.List;

class Library {
    private List<String> titles = new ArrayList<>();

    void addTitle(String title) {
        titles.add(title);
    }

    void printTitles() {
        for (String t : titles) {
            System.out.println(t);
        }
    }
}

public class BooksOop {
    public static void main(String[] args) {
        Library library = new Library();
        library.addTitle("Java Basics");
        library.addTitle("Data Structures");
    }
}
```

```
        library.printTitles();
    }
}
```

The procedural version keeps the data in a static array shared by static methods. The object-oriented version wraps both the data and the behavior (adding and printing titles) inside a Library object, allowing multiple independent Library instances to exist, each managing its own list of titles.

Q3. What are the strengths and weaknesses of declarative vs imperative paradigms?

Declarative paradigm:

- Strength: code expresses what result is wanted, which is often shorter, closer to the problem statement, and easier to reason about.
- Strength: the underlying system (database engine, functional runtime) can often optimize how the result is produced.
- Weakness: less direct control over exact execution steps, which can make performance tuning harder.
- Weakness: can have a steeper learning curve for programmers used to thinking in terms of explicit steps.

Imperative paradigm:

- Strength: gives explicit, fine-grained control over how a task is performed, including control over performance and memory use.
- Strength: often more intuitive for beginners, since it mirrors step-by-step, real-world instructions.
- Weakness: code can become longer and more detailed, since every step must be spelled out.
- Weakness: mutable shared state can make larger imperative programs harder to reason about and more prone to bugs, especially in concurrent contexts.

In practice, many modern languages, including Java, blend both paradigms so programmers can choose declarative constructs (such as streams) where clarity matters most, and imperative constructs where fine control is needed.